Lesson1_2_6

September 4, 2026

1 Introduction

This web site is running a "Jupyter notebook," which allows you to execute interactive Octave code directly in your browser. Each (part of a code) line preceeded by a percent symbol "%" is a code comment.

To execute any Octave commands, first enter the commands in a code cell, and then depress the "shift" and "enter" keys on your keyboard at the same time, making sure that your cursor is placed inside of the code cell. I refer to this as "< shift >< enter >" in my instructions below.

The output from this notebook will be used to provide answers for this lesson's practice quiz. As you proceed through the specialization, you will use more and more Jupyter notebooks to write Octave code to implement Kalman-filter algorithms.

[1]:
```
% Place your cursor in this input code cell. Then, type < shift >< enter > to␣
 ↪execute

% Uncomment the next line if you wish to experiment with the "ss" and "lsim"␣
 ↪commands, which
% require loading the Octave control-systems "package."
% pkg load control

maxT = 1000; % maximum simulation time, used for all models (in iterations)
dT = 0.1;    % sample period used by all models (in seconds)
```

1.0.1 The nearly constant position model

The next notebook cells demonstrate the nearly constant position model

[2]:
```
% Define the NCP model
Ancp = eye(2);
Bwncp = eye(2);
Cncp = eye(2);
Dncp  = zeros(2);
```

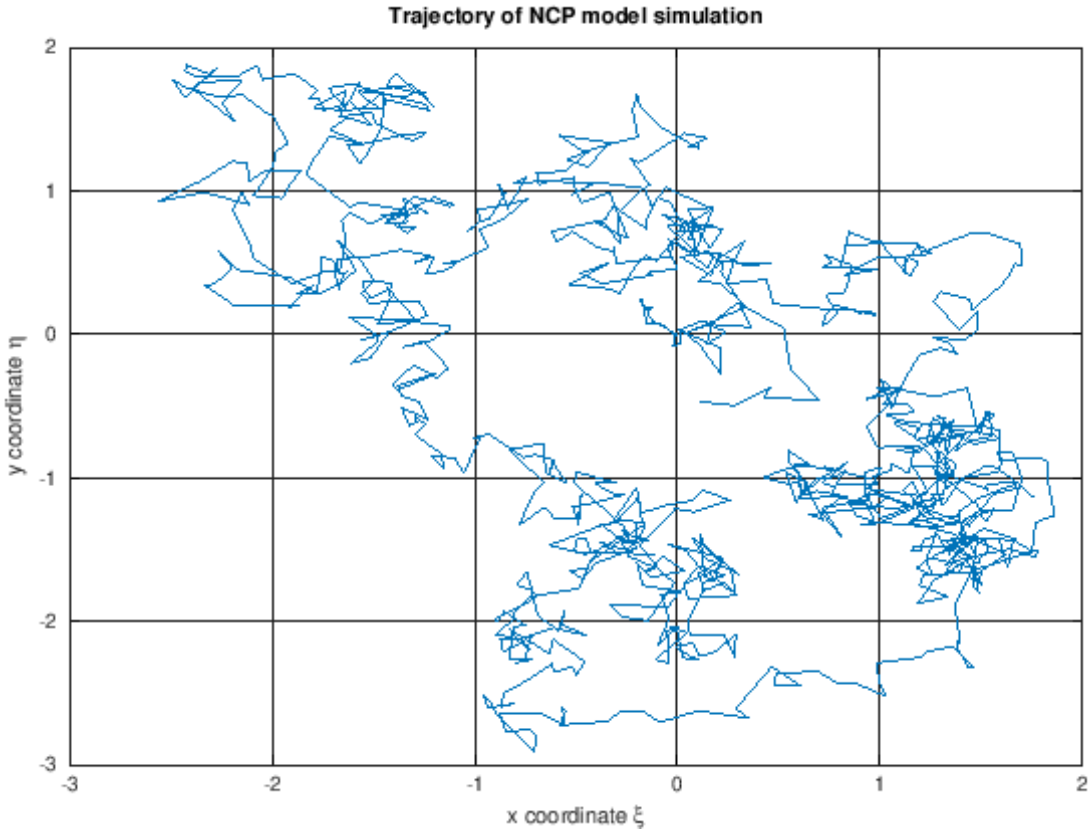
[3]:
```

```

```
% Simulate the NCP model
randn("state",0); % Don't change this line since it affects the simulation␣
 ↪results that are used in the quiz
wncp = 0.1*randn(2,maxT);
vncp = 0.01*randn(2,maxT);

xncp = zeros(2,maxT); % storage
xncp(:,1)=[0;0];    % initial posn.
for k=2:maxT     % simulate model
  xncp(:,k)=Ancp*xncp(:,k-1)+Bwncp*wncp(:,k-1);
end
zncp = Cncp*xncp + vncp;
```

[4]:
```
% Plot the simulation output
plot(zncp(1,:),zncp(2,:));
xlabel('x coordinate \xi');
ylabel('y coordinate \eta');
title('Trajectory of NCP model simulation');
grid on
```



[5]:
```
% Display the final coordinate of the trajectory
zncp(:,end)
```

```
ans =

   0.11226
  -0.46821
```

1.0.2 The nearly constant velocity model

The next notebook cells demonstrate the nearly constant velocity model

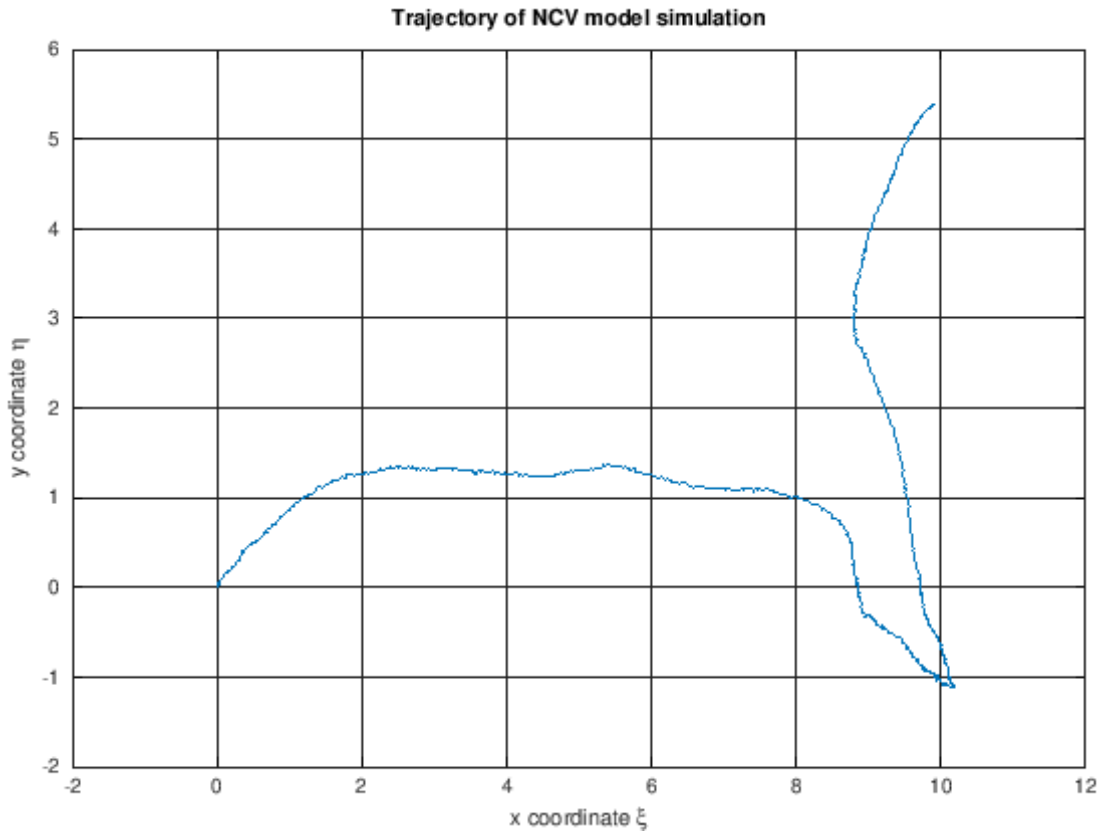
[6]:
```
% Define the NCV model
Ancv = [1 dT 0 0; 0 1 0 0; 0 0 1 dT; 0 0 0 1];
Bwncv = [dT^2/2 0; dT 0; 0 dT^2/2; 0 dT];
Cncv = [1 0 0 0; 0 0 1 0];
Dncv = zeros(2);
```

[7]:
```
% Simulate the NCV model
randn("state",0); % Don't change this line since it affects the simulation␣
 ↪results that are used in the quiz
wncv = 0.1*randn(2,maxT);
vncv = 0.01*randn(2,maxT);

xncv = zeros(4,maxT); % storage
xncv(:,1) = [0;0.1;0;0.1]; % initial position, velocity

for k=2:maxT    % simulate model
  xncv(:,k)=Ancv*xncv(:,k-1)+Bwncv*wncv(:,k-1);
end
zncv = Cncv*xncv + vncv;
```

[8]:
```
% Plot the simulation output
plot(zncv(1,:),zncv(2,:));
xlabel('x coordinate \xi');
ylabel('y coordinate \eta');
title('Trajectory of NCV model simulation');
grid on
```



[9]:
```
% Display the final coordinate of the trajectory
zncv(:,end)
```

```
ans =

   9.8816
   5.3740
```

1.0.3 The coordinated-turn model

The next notebook cells demonstrate the coordinated-turn model

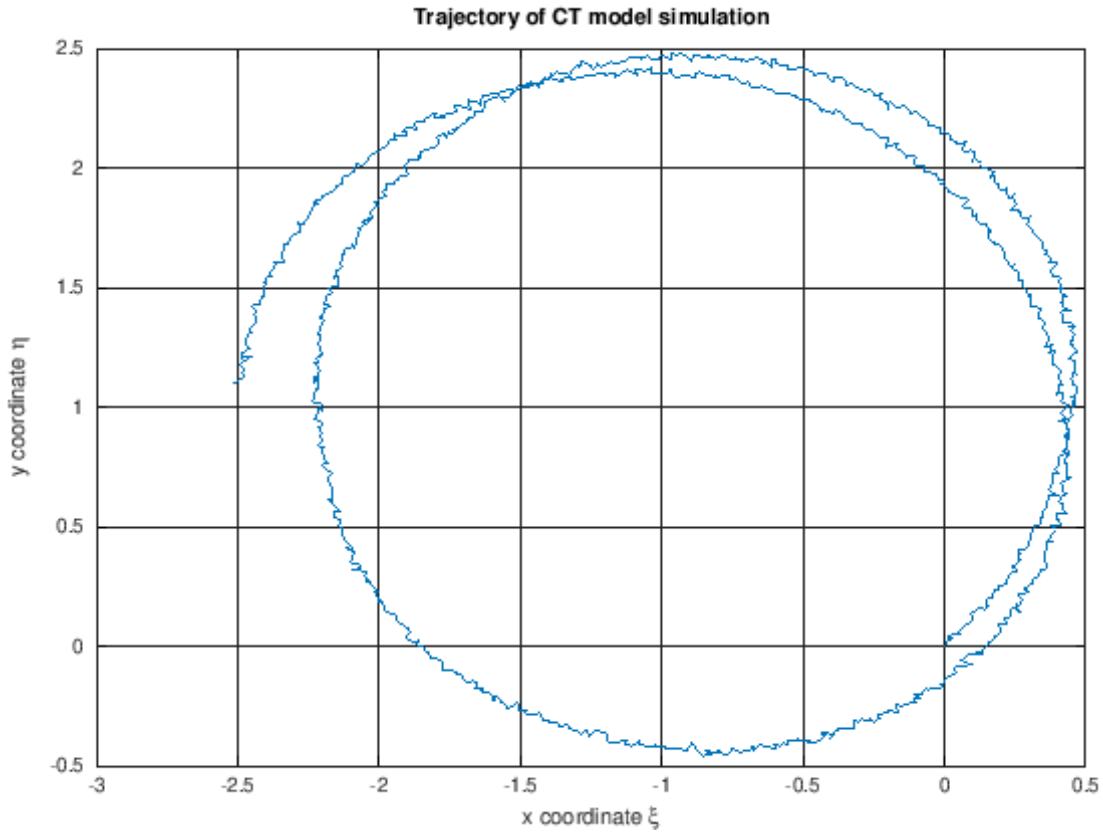
[10]:
```
% Define the CT model
W = 0.1; WT = W*dT; % Use W as Omega
sW =sin(WT); cW = cos(WT);
Act  = [1 sW/W 0 (cW-1)/W; 0 cW 0 -sW; 0 (1-cW)/W 1 sW/W; 0 sW 0 cW];
Bwct = [(1-cW)/W^2, (sW-WT)/W^2; sW/W (cW-1)/W; (WT-sW)/W^2, (1-cW)/W^2; (1-cW)/W sW/W];
Cct = [1 0 0 0; 0 0 1 0];
```

```
Dct = zeros(2);
```

[11]:
```
% Simulate the CT model
randn("state",0); % Don't change this line since it affects the simulation␣
 ↪results that are used in the quiz
wct = 0.01*randn(2,maxT);
vct = 0.01*randn(2,maxT);
xct = zeros(4,maxT);      % reserve storage
xct(:,1) = [0;0.1;0;0.1]; % initial posn, velocity

for k=2:maxT    % simulate model
  xct(:,k)=Act*xct(:,k-1)+Bwct*wct(:,k-1);
end
zct = Cct*xct + vct;
```

[12]:
```
% Plot the output of the CT model
plot(zct(1,:),zct(2,:));
xlabel('x coordinate \xi');
ylabel('y coordinate \eta');
title('Trajectory of CT model simulation');
grid on
```



[13]:
```
% Display the final coordinate of the trajectory
zct(:,end)
```

```
ans =

  -2.5114
   1.1001
```